using its own component runtime. The unit of deployment was a plug-in (just a JAR file, as any OSGi bundle). Each plug-in provided extension points, using which other bundles could extend the behaviour of the original module. Plug-ins used *plugin.xml* file for declaring extension points and other metadata. After Equinox adoption, a compatibility layer was built to let pre-3.0 implementations continue to work without rewriting. Some Eclipse-specific metadata remained in *plugin.xml*, and others like bundle name, version, export-package, etc, moved to the bundle manifest file. The plug-in development environment (PDE) support was added in Eclipse to let developers extend Eclipse with their own plug-in. In the *plug-ins* folder of Eclipse, there would be a JAR with the name *org.eclipse.osgi.<version_number>* and this is the Equinox runtime that runs Eclipse and every other plug-in built by developers. There are many optional and mandatory bundles implementing key features of OSGi, like security, logging, updates, preferences, etc. The Eclipse workbench is built using the Standard Widget Toolkit (SWT) and JFace, which provides a native look and feel. With an extensible plug-in mechanism and a rich cross-platform look and feel, Eclipse, which started as an integrated development environment (IDE), soon became a platform for rich client applications. In fact, it is called a rich client platform (RCP).

Tooling for Equinox

Theoretically, one would not need more than the JDK and a text editor to develop an Equinox bundle—but that would not be a productive approach. By selecting the right tools, any standard IDE like Eclipse or NetBeans can be equipped for Equinox development. Some of them are Bnd, Pax Runner, Pax Exam and Eclipse PDE itself.

Bnd

Bnd, as the creator claims, is the Swiss army knife of OSGi development. It is used for creating and manipulating OSGi bundles. It takes a .bnd file as input, and creates OSGi bundles. It makes bundle development easier by providing many conveniences. For example, a bundle can export or import multiple packages, and this has to be declared in the manifest file. If hand-coding the manifest, the developer has to list all the packages, but the *.bnd* specification file accepts wild-cards in package names, and creates the bundle manifest accordingly. Bnd can also do dependency analysis and update the manifest. Bnd can build bundles from sources available on the file system, or other JAR files, as specified in the *.bnd* file. The bundle JAR can be verified for specification consistency by running Bnd's *verify* command. Bnd is available as an IDE plug-in, and also as Ant and Maven tasks. More information and downloads are available at *http://www.aqute.biz/Bnd/Bnd*

Pax Runner

Pax Runner makes switching between multiple open source implementations of OSGi easier. The developer can focus on building bundles instead of learning how to start and deploy bundles in multiple OSGi implementations. With a command-line switch, Pax Runner can be made to start different OSGi implementations. It also allows loading bundles from multiple sources like the URL, file system, Zip file, Maven repository, etc. Bundles can be grouped as profiles, and can be deployed or un-deployed collectively. Pax Runner is available as a standalone tool, and also as an Eclipse plug-in. More details at *http://team.ops4j.org/wiki/display/paxrunner/Pax+Runner*

Eclipse PDE

The Eclipse Plug-in Development Environment built on top of the JDT can also be used for building OSGi bundles, but is more suitable for the development of Eclipse plug-ins than OSGi bundles. For example, dependency management between bundles is handled differently in plug-ins and bundles. The Eclipse plug-in prefers the *require-bundle* directive, whereas an OSGi bundle uses the *import-package* directive. Eclipse is one of the most preferred Java development environments, and hence Eclipse with the *bnd* plug-in would make a near-ideal environment for OSGi bundle development.

Tycho

Tycho is a Maven-centric OSGi bundle development tool currently in the incubation phase. Tycho supports building bundles, plug-ins, fragments, RCP applications and update sites. Further details can be found at *http://eclipse.org/tycho/documentation.php*.

OSGi for Web applications

The traditional Web application development process was not dynamic, as addition of new features involved redeployment, resulting in unavoidable downtime. OSGi for Web applications was sought as a solution to this problem. OSGi, being a dynamic module system, can reduce or avoid downtime by dynamically deploying or un-deploying modules. As an added benefit, application complexity is reduced with loosely coupled modules.

Deployment of an OSGi-based Web application is possible in two ways:

- Embedding an HTTP server in Equinox
- Embedding Equinox in an existing application container

Embedding an HTTP server in Equinox

Equinox provides bundles that implement a basic HTTP service, and in simple terms, a minimal application server or servlet container. Currently, there are two different implementations providing different levels of servlet support.

The single-bundle implementation *org.eclipse.equinox.http* is suited for resource-constrained environments. Although the API is compatible with Servlet 2.4, this implementation offers limited support above Servlet 2.1.